

TPU-Lite-SoC 设计报告

姓名：包镭嘉，段博怀，胡继永，王嘉瑞，徐若航，朱展彤

2026 年 1 月 9 日

1 TPU-Lite-SoC 概述

1.1 项目概述

TPU-Lite-SoC 是一款面向轻量级深度学习推理任务的专用处理器片上系统 (System-on-Chip)，可实现 MNIST 数字分类等低复杂度神经网络。该项目通过软硬件协同设计，实现了高效的 INT8 精度推理，兼顾计算性能与硬件资源占用。

1.2 核心架构组成

项目架构覆盖硬件模块设计、软件工具链开发及系统集成三大维度，关键组成如下：

- **TPU 核心模块**：包含统一缓冲区 (Unified Buffer)、4 个的 16×16 脉动阵列 (Systolic Array)、多通道向量处理单元 (VPU) 及控制单元，支持矩阵乘、偏置加、ReLU 激活、反量化等神经网络核心操作；
- **SoC 集成模块**：采用开源 RISC-V CPU 作为主机 CPU，配置主存，集成基于 iDMA 的数据搬运引擎，通过 64-bit AXI 总线实现各模块间高效数据交互；
- **软件工具链**：基于 PyTorch 完成 MLP 模型训练与 INT8 量化，开发与硬件全流程对齐的推理仿真器，提供指令设计所需的计算流程拆分与接口规范。

1.3 关键技术特性

1. 可编程性：通过 54-bit 指令集实现对 Unified Buffer、脉动阵列、VPU 的精细化控制，支持灵活的计算流程配置；2. 并行计算：4 个脉动阵列共享输入数据，可同时处理多组权重，VPU 采用 4 通道并行架构，提升批量数据处理效率；3. 高效数据传输：DMA 引擎支持批量数据的并行搬运，相较于 CPU 直接传输，最大可获得 $14.68\times$ 的性能提升；4. 高精度推理：INT8 量化方案保证模型精度损失可控，推理准确率可达 97.56%。

1.4 项目价值与应用场景

TPU-Lite-SoC 实现了轻量级神经网络推理的软硬件协同设计，其低资源占用、高计算效率的特性适用于嵌入式 AI、边缘计算等场景。项目验证了专用处理器架构在低复杂度深度学习任务中的优势，为后续高性能、低功耗 AI 芯片设计提供了可复用的架构方案与验证方法。

1.5 设计报告内容安排

在章节2中，介绍 TPU-Lite 的顶层架构，编程模型与控制逻辑，在章节3中，介绍 Systolic array 的架构设计与实现，在章节4中，介绍 VPU 的架构设计与实现，在章节5中，介绍 SoC 系统集成与 DMA Engine 实现。在章节6中，介绍 TPU-Lite-SoC 软件层设计与实现。在章节7中，介绍 TPU-Lite-SoC 的功能实现与验证。

1.6 小组分工

小组分工如表1所示。

姓名	负责内容
包裕嘉	TPU-Lite 控制逻辑与存储单元设计与实现
段博怀	TPU-Lite-SoC 软件层设计与实现
胡继永	VPU 架构设计与实现
王嘉瑞	Systolic array 架构设计与实现
徐若航	TPU-Lite 顶层架构实现与验证
朱展彤	DMA Engine 设计，SoC 集成与验证

表 1: 小组成员分工

2 TPU 内部存储与控制模块

本节将介绍 TPU 的编程模型及其硬件实现，首先定义软件如何操控 TPU 硬件，随后逐模块介绍硬件设计，最后展示仿真验证结果。

2.1 TPU 编程模型

TPU 内部的数据流和控制流如图1所示。复位后，状态寄存器中 `global_en` 位为 0，TPU 不工作。在 TPU 开始工作前，SoC 通过 AXI 总线将所需的全部数据写入 Global Buffer，将所需的全部指令写入 Instruction Cache。

当数据和指令均写入完毕后，SoC 通过 AXI 总线向状态寄存器中 `global_en` 位写入 1，使 TPU 开始工作。工作时，每个时钟周期 Control Unit 从 Instruction Cache 中读出一条指令，解码出不同的字段并将其转为 Global buffer、Systolic Array 和 VPU 所需的控制信号，控制 Global Buffer 向 Systolic Array 输出数据和 VPU 向 Global Buffer 写回数据。为了适配 Systolic Array 的输出形式，VPU 中不同 Channel 需要异步操作，这部分数据流由 VPU 内部控制，不需要 Control Unit 提供控制信号。在执行到最后一条有效指令后，Control Unit 将状态寄存器中 `finish` 位写入 1，并停止 TPU 工作。随后，SoC 在检测到状态寄存器中 `finish` 位被置为 1 后，通过 AXI 总线从 Global Buffer 中读出结果。

如上所述，TPU 的控制流很大程度受软件控制，因此具备一定可编程性。下面将介绍 TPU 内存编址和指令定义。

2.1.1 内存编址

TPU 中有 5 块内存可被 AXI 总线访问，其中前三块内存（Weight Memory、Input/Psum Memory 和 Scale/Bias Memory）都属于 Gloabl Buffer。这些内存的 AXI 接口被映射到同一个地址空间，以 64-bit 为访问粒度连续编址，如图2所示。

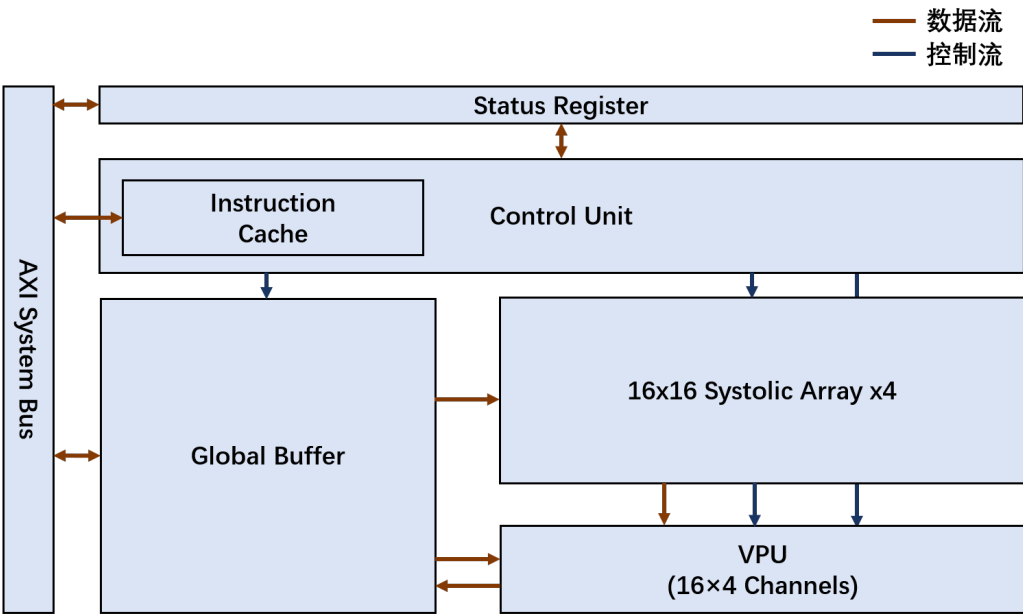


图 1: TPU 内部数据流及控制流

地址范围	存储器	读写	内部实现
0x0000 ~ 0x1FFF	Weight Memory	只写	1024×512bit
0x2000 ~ 0x27FF	Input/psum Memory	读写	256×512bit
0x2800 ~ 0x29FF	Scale/Bias Memory	只写	16×2048bit
0x2A00 ~ 0x2DFF	Instruction Cache	只写	1024×54bit
0x2E00	Status Register - global_en	只写	1bit
0x2E01	Status Register - finish	只读	1bit
所有地址都支持AXI 64-bit粒度访问			

图 2: TPU 内存编址

2.1.2 指令定义

TPU 指令定义如图3所示。单条指令长度为 54-bit，分为四类字段，分别用于控制 Global Buffer、Systolic Array、VPU 和全局状态。

Gloabl Buffer 控制部分包含 7 个字段，其中 5 个字段分别控制 input/weight/bias/scale 的读出地址和 VPU 写回地址。以上字段中的地址均采用各块内存内部的地址，从而节约指令位宽。Array 使能数量字段控制 weight 和 bias 的读出位宽；VPU 写使能数量字段控制 VPU 写回的位宽。由于 4 个 Systolic Array 共享 Input，故 input 读出位宽固定；另外所有的 VPU channel 共享 scale，故 scale 的读出位宽也固定，不需要额外字段控制。

Systolic Array 控制部分包含 4 个字段，具体功能详见 Systolic Array 一节。

VPU 控制部分包含 5 个字段，具体功能详见 VPU 一节。

全局控制字段部分仅包含 1 个字段。若该位为 1，则表明 VPU 任务结束，控制单元不再继续读出指令，并使 Global Buffer/Systolic Array/VPU 等模块均停止工作。因此，只有最后一条有效指令的此位应当为 1，其余指令的此位都应该为 0。

2.2 模块设计

TPU 内部存储与控制涉及以下模块：

- AXI interface

GB控制字段							
字段地址	[9:0]	[19:10]	[31:20]	[35:32]	[39:36]	[41:40]	[43:42]
字段位置	10	10	12	4	4	2	2
字段内容	input读出地址	VPU写回地址	weight读出地址	bias读出地址	scale读出地址	Array使能数量	VPU写回使能数量
字段说明	input mem内地址 0x000~0x3FF	input mem内地址 0x000~0x3FF	weight mem内地址 0x000~0xFFF	misc mem内地址 0x0~0xF	misc mem内地址 0x0~0xF	0: 0个 1: 1个 2: 2个 3: 4个	0: 0个 1: 1个 2: 2个 3: 4个

SA控制字段			VPU控制字段				全局控制字段	
字段地址	[44]	[45]	[48:47]	[49]	[50]	[51]	[52]	[53]
字段位置	1	1	2	1	1	1	1	1
字段内容	Array写入新权重	Array内Buffer翻转	Array输入有效	VPU模式	是否清零psum	是否应用bias	是否应用relu	是否应用量化
字段说明	0: 不写入 1: 写入	0: 不翻转 1: 翻转	0: 输入无效(31周期后 计算结束) 1: 输入有效	0: Idle模式 1: Accu模式 2: Load模式(不支持) 3: Output模式	0: 不清零 1: 清零, 随后进入Acc 模式	0: 不应用 1: 应用	0: 不应用 1: 应用	0: 尚未结束 1: 结束

图 3: TPU 指令定义

- Status Register
- Global Buffer
- Systolic Data Re-arranger
- Instruction Cache
- Control Unit

下面将逐模块介绍其实现与核心代码：

2.2.1 AXI interface

该模块根据 AXI 信号选中内部模块并转换地址。以写入 Global buffer 为例，核心代码如代码 1 所示，根据地址判断内部某模块是否被选中，如被选中，则将地址变换为内部地址，并传递数据和使能信号，否则不使能该模块的读写。

2.2.2 Status Register

该模块用于控制及标识 TPU 全局状态，包含 2 位。其中 gloabl_en 位只可被 AXI 总线写入，用于在数据及指令准备完毕后启动 TPU；finish 位虽然实现了 AXI 写入的接口，但是正常使用时应只被 control unit 写入，用于标识 TPU 计算完成，如代码2 所示。

2.2.3 Global Buffer

该模块用于存储 TPU 工作过程中涉及的各类数据，包含 Systolic Array 处理矩阵乘时所需的 Weight 和 Input，以及 VPU 处理向量时所需的 Bias 和 Scale。在某一层计算完成后，VPU 输出结果也写回该模块。实现时，模块内部分为 3 块内存，分别是

- Weight memory
容量为 512 Kb，支持 128/256/512 bit per cycle 读出 (16row×8bit×1/2/4array)
- Input/Partial Sum memory
容量为 128 Kb，支持 128 bit per cycle 读出，128/256/512 bit per cycle 写入
- Bias/Scale memory
容量为 32 Kb，支持 32/512/1024/2048 bit per cycle 读出

```

1 // Generate internal address
2 always_comb begin
3     addr_in_TPU = 16'hFFFF;
4     if (axi_addr >= TPU_BASE && axi_addr < TPU_END) begin
5         addr_in_TPU = axi_addr[18:3];
6     end
7 end
8 // 写控制
9 always_comb begin // UBUF
10     ubuf_en      = 1'b0;
11     ubuf_we      = 1'b0;
12     ubuf_addr    = 16'b0;
13     ubuf_wdata   = 64'b0;
14     if (axi_req && (addr_in_TPU >= UBUF_BASE) && (addr_in_TPU <= UBUF_END)) begin
15         ubuf_en = 1'b1;
16         ubuf_addr = addr_in_TPU - UBUF_BASE;
17         if (axi_we) begin
18             ubuf_we = axi_we;
19             ubuf_wdata = axi_wdata;
20         end
21     end
22 end

```

Listing 1: AXI Interface 模块核心代码（以写入 Global Buffer 为例）

```

1 logic reg_global_en; // AXI 只写寄存器
2 logic reg_finish; // AXI 只读寄存器
3 always_ff @(posedge clk or posedge rst) begin
4     if (rst) begin
5         reg_global_en <= 1'b0;
6         reg_finish <= 1'b0;
7     end else begin
8         if(axi_status_en && axi_status_we && (axi_status_addr == ADDR_REG_EN)) begin
9             reg_global_en <= axi_status_wdata[0];
10        end
11        reg_finish <= ctrl_finish_in;
12    end
13 end
14 assign global_en = reg_global_en;

```

Listing 2: Status Register 模块核心代码

每块内存的单行位宽都根据其最大可能读出/写入位宽来设定。其中, Weight memory 和 Input/Partial Sum 的单行位宽都为 512-bit, 但完整支持 128/256/512bit 粒度操作。例如, 在 128-bit/cycle 读出模式下, Weight memory 可被视为一块 4096×128 -bit 的内存; 在 256-bit/cycle 读出模式下, Weight memory 可被视为一块 2048×256 -bit 的内存, 地址的最低 1 位会被自动舍掉; 在 512-bit/cycle 读出模式下, Weight memory 可被视为一块 1024×512 -bit 的内存, 地址的最低 2 位会被自动舍掉。如代码 3 所示。

而 Bias/Scale memory 由于操作位宽跨度过大 (32-bit 至 2048-bit), 故方便起见, 实现为 16×2048 bit, 读取位宽低于单行位宽时, 截取行内低比特。

以上内存均支持 AXI 接口以 64-bit 为粒度进行写入。

2.2.4 Systolic Data Re-arranger

该模块用于将 Global Buffer 中整行读出的数据以 Systolic Array 所需的时序延迟 0 到 15 个周期输出。这一功能通过每行放置不同深度的 FIFO 实现。第 1 行的延迟为 0, 故输入直接连到输出, 后续第 i 行的输入在经过 $i-1$ 个 DFF 后输出, 故延迟了 $i-1$ 拍。

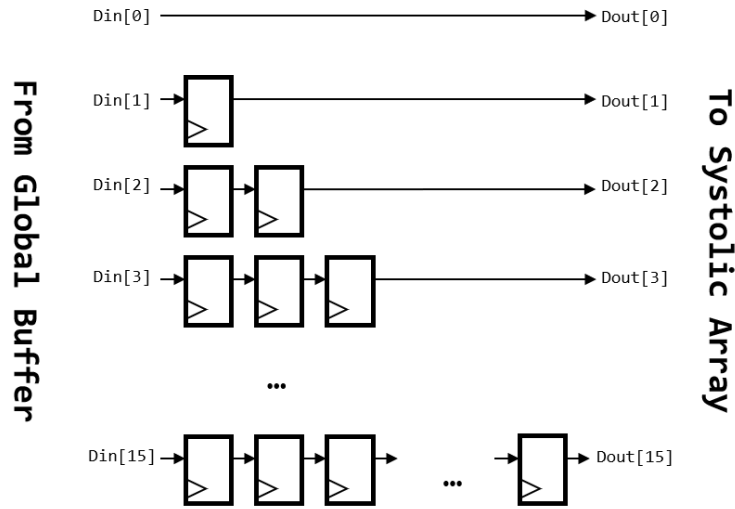


图 4: Systolic Data Re-arranger 结构

2.2.5 Instruction Cache

该模块用于存储控制 TPU 工作的指令, 大小为 54 Kb。该内存只可被 AXI 写入, 写入时自动截取低字节; 正常情况下, 该模块只被 control unit 读出, 读出位宽为 54-bit。本模块核心代码如代码 4 所示。为便于验证, 在仿真时, 该模块可以直接通过 \$readmemb 用文件来初始化。

2.2.6 Control Unit

该模块用于读出指令并将其解码为各个模块所需的控制信号。当状态寄存器中 global_en 位为 0 时, 该模块不读出指令, 并将所有控制信号设为不使能的状态。当状态寄存器中 global_en 位为 1 时, 该模块读出一条指令并将 instruction cache 的访问地址加一, 同时根据读出的指令改变 TPU 中各模块的控制信号, 直到读出指令中最高位 (即全局 finish 字段) 为 1。

2.3 仿真验证结果

本部分将展示 TPU 内部存储与控制模块的三项主要功能的仿真结果, 并对其进行简单分析。

```

1 // Weight 读出逻辑
2 // 解析读出地址到weight存储坐标
3 logic [9:0]      sa_wmem_rd_row;
4 logic [1:0]      sa_wmem_inrow_offset_128;
5 logic           sa_wmem_inrow_offset_256;
6 assign sa_wmem_rd_row      = ub_rd_weight_addr_in[11:2];
7 assign sa_wmem_inrow_offset_128 = ub_rd_weight_addr_in[1:0];
8 assign sa_wmem_inrow_offset_256 = ub_rd_weight_addr_in[1];
9 logic [127:0]  ub_rd_weight_data_out_temp [3:0]; // 用于格式转换
10 always_comb begin
11     for (int i = 0; i < 4; i++) begin
12         ub_rd_weight_data_out_temp[i] = 128'b0;
13     end
14     if(ub_rd_weight_en) begin
15         case(ub_rd_weight_size_in)
16             2'b01: begin
17                 ub_rd_weight_data_out_temp[0] = weight_memory[sa_wmem_rd_row][
18                     sa_wmem_inrow_offset_128*128 +: 128];
19             end
20             2'b10: begin
21                 ub_rd_weight_data_out_temp[0] = weight_memory[sa_wmem_rd_row][
22                     sa_wmem_inrow_offset_256*128 +: 128];
23                 ub_rd_weight_data_out_temp[1] = weight_memory[sa_wmem_rd_row][
24                     sa_wmem_inrow_offset_256*256 +: 128];
25             end
26             2'b11: begin
27                 ub_rd_weight_data_out_temp[0] = weight_memory[sa_wmem_rd_row][511:384];
28                 ub_rd_weight_data_out_temp[1] = weight_memory[sa_wmem_rd_row][383:256];
29                 ub_rd_weight_data_out_temp[2] = weight_memory[sa_wmem_rd_row][255:128];
30                 ub_rd_weight_data_out_temp[3] = weight_memory[sa_wmem_rd_row][127:0];
31             end
32         endcase
33     end
34 end
35 genvar i_rd_weight, j_rd_weight;
36 generate
37     for (i_rd_weight = 0; i_rd_weight < 4; i_rd_weight = i_rd_weight+1) begin
38         for (j_rd_weight=0; j_rd_weight < 16; j_rd_weight = j_rd_weight+1) begin
39             assign ub_rd_weight_data_out[i_rd_weight][15-j_rd_weight] =
40                 ub_rd_weight_data_out_temp[i_rd_weight][j_rd_weight*8 +: 8];
41         end
42     end
43 endgenerate

```

Listing 3: Global Buffer 核心代码

```

1 logic [INS_LEN-1:0] ins_memory [1023:0];
2 // AXI 写入逻辑
3 logic [9:0] axi_insmem_wr_row;
4 assign axi_insmem_wr_row = axi_icache_addr[9:0];
5 always_ff @(posedge clk or posedge rst) begin
6     if (rst) begin
7         `ifdef LOAD_TXT
8             $readmemb("../instruction/ins.txt", ins_memory);
9         `else
10            for (int i = 0; i < 1024; i = i + 1) begin
11                ins_memory[i] <= 'b0;
12            end
13        `endif
14    end else begin
15        if (axi_icache_en && axi_icache_we) begin
16            ins_memory[axi_insmem_wr_row] <= axi_icache_wdata[INS_LEN-1:0];
17        end
18    end
19 end
20 // Ctrl 读出逻辑
21 always_comb begin
22     if(rst) begin
23         icache_rd_ctrl_data = 'b0;
24     end else if(icache_rd_ctrl_en) begin
25         icache_rd_ctrl_data = ins_memory[icache_rd_ctrl_addr];
26     end else begin
27         icache_rd_ctrl_data = 'b0;
28     end
29 end

```

Listing 4: Instruction Cache 模块核心代码

2.3.1 SoC 写入 TPU 功能仿真

首先验证 SoC 写入 TPU 功能，先向 Global Buffer 内 Weight Memroy 的两个相邻地址（对应内部访问时一个 128-bit 数的高低字节）写入 0xAA01 和 0xBB02；随后向 Global Buffer 内 Input/Partial Sum memory 内的一个任意地址写入 0xCC03；再向 Global Buffer 内 Bias/Scale memory 内的一个任意地址写入 0xDD04；然后向 Instruction Cache 内的一个任意地址写入 0xEE05；最后向状态寄存器中 global_en 位写入 1。如图5所示，各内存均被正常写入，说明此功能实现正确。

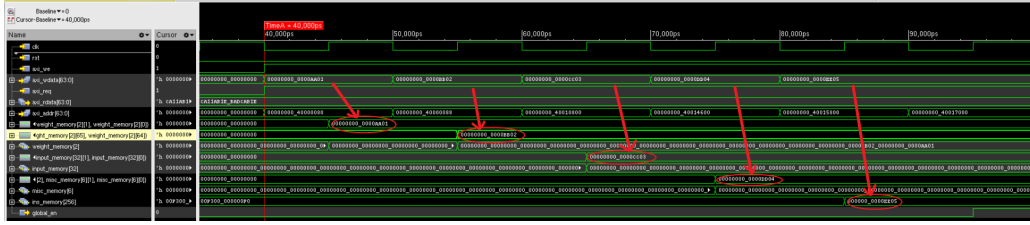


图 5: 控制单元仿真结果

2.3.2 控制单元功能仿真

控制单元的仿真结果如图6 所示，在状态寄存器中 global_en 位被至高后，control unit 使 instruction cache 的访问地址每周周期增加 1，并解码读出指令中的各个字段，将其转换为各模块需要的控制信号，表明控制单元功能正常。

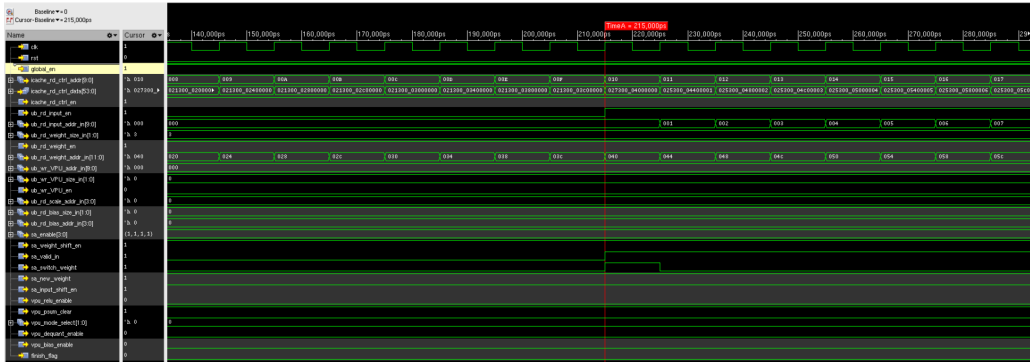


图 6: 控制单元仿真结果

2.3.3 Global Buffer 内部端口功能仿真

如图7 所示，Global Buffer 接收 Control Unit 解码出的地址，从内存中对应位置整行读出数据，随后 Systolic Data Re-arranger 将其延迟 0 到 15 个周期输出至 Systolic Array，行为与预期一致，表明 Global Buffer 内部端口功能正常。

3 Systolic Array

3.1 系统设计

系统整体结构如图8所示，Systolic Array 接收 Control Unit 发送而来的 valid_in 等信号，对从 Unified Buffer 获取的 input 与 weight 数据进行脉动阵列计算，并将结果（包含数据结果和输出有效信号）输送给 VPU，实现完整的计算功能。



图 7: 阵列及排序仿真结果

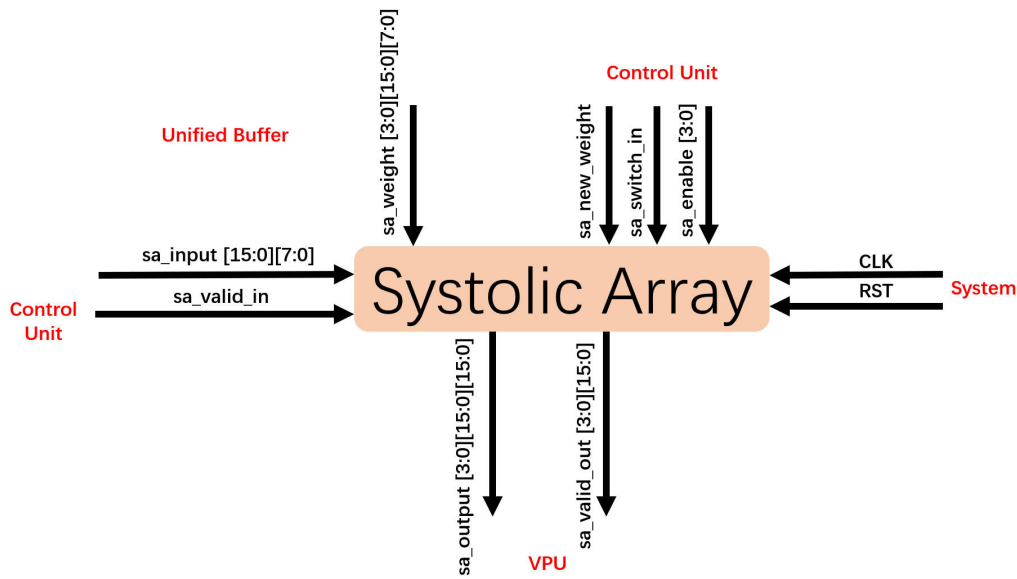


图 8: 4x16x16 Systolic Array 系统框图

3.1.1 信号说明

系统主要信号如下：

- **sa_valid_in**: 输入有效信号（由左上角单元接收，按对角线方向传播）
- **sa_input [15:0][7:0]**: 输入数据总线
- **sa_weight [3:0][15:0][7:0]**: 权重数据总线
- **sa_new_weight**: 新权重标志，控制 Background Weight Buffer 中的权重移动行为
- **sa_switch_in**: 控制 Foreground Weight Buffer 接收 Background Weight Buffer 中的权重（由左上角单元接收，按对角线方向传播）
- **sa_valid_out [3:0][15:0]**: 输出有效信号，每列一个
- **sa_output [3:0][15:0][15:0]**: 输出数据总线
- **sa_enable [3:0]**: 阵列使能信号

```

1 always_ff @(posedge clk or posedge rst) begin
2     if (rst) begin
3         for (int j = 0; j < 16; j++) begin
4             col_new_weight[j] <= 1'b0;
5         end
6     end else begin
7         for (int j = 0; j < 16; j++) begin
8             col_new_weight[j] <= (j == 0) ? sys_new_weight : col_new_weight[j-1];
9         end
10    end
11 end
12 // .....
13 assign local_valid_w_in = (col_new_weight[j]) ? 0 : pe_valid_w_out[i-1][j];

```

Listing 5: Systolic Array 权重切换机制代码

3.1.2 权重切换机制

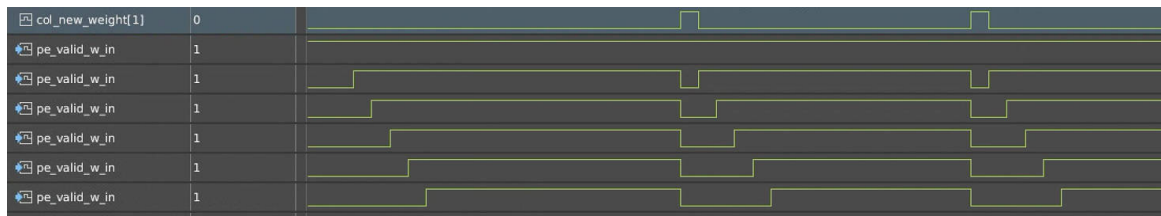
为实现最高计算效率，权重从 Background 切换到 Foreground 的过程与输入数据沿对角线方向同步传播，由切换信号 `switch_in` 控制，沿对角线方向传递。

另外，由于权重切换是由左上至右下的过程，需要确保还未切换的权重保持原位，不会随新权重输入下移。当某一列的 `col_new_weight` 信号为高电平时，代表该列即将接收新的权重矩阵，之前权重矩阵已经输入完成，此时应停止该列权重继续向下移动（首行仍然接收新权重）。在逻辑实现上，让 `col_new_weight` 信号上跳沿控制 `valid_w_in` 置 0（首行除外），后续权重与 `valid_w_in` 信号一同逐行下传（如代码5所示）。

3.2 仿真与结果分析

在仿真中，使能了 Array1 和 Array2，并为其分别配置不同的权重数据，以测试多 Array 同时计算的可行性及片选功能的正确性。在时间维度上，构造单位矩阵和全 1 矩阵两个输入矩阵，紧密相接输入，观测四个阵列的输出结果：

- 权重切换机制中的 `valid_w_in` 信号符合预期，在 `col_new_weight` 信号上跳的同时变为 0，并逐行传递变回 1（如图9）

图 9: `valid_w_in` 信号仿真结果

- 未使能的阵列（Array3 和 Array4）输出 `valid` 和 `output` 均为零，符合预期（如图10）。
- 使能的阵列输出 `valid` 和 `output` 均与预期一致（如图11），验证了权重切换与数据同步机制的正确性。



图 10: valid_out 信号仿真结果

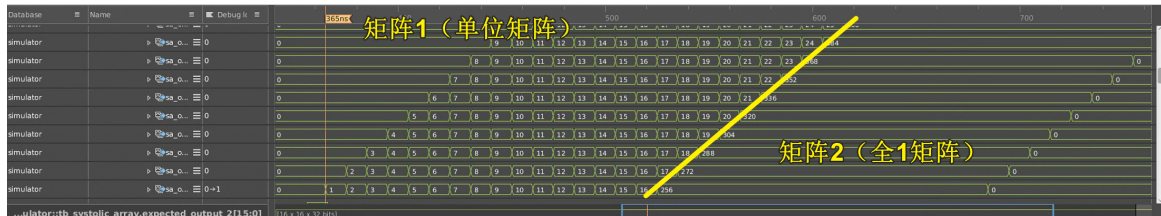


图 11: Systolic Array 输出计算结果仿真波形

3.3 Systolic Array 部分总结与展望

本次设计成功将 Systolic Array 扩展为四个相对独立的部分，提升了系统灵活性。四个阵列共享输入，可同时处理多组不同权重，适用于并行计算场景。未来可进一步优化系统以支持同时处理多个输入数据流，增强其在实时推理与训练任务中的应用能力。

4 VPU 部分设计与仿真

本项目中，VPU 主要担任以下的任务：

1. 多批次 Partial Sum 累加
2. 添加偏置 bias
3. 非线性函数 (ReLU)
4. 反量化

以上 4 个任务在每一轮运算中会选择若干个通过。例如：如果存在多个 tile 的需要累加，需要通过 PSUM_CACHE 模块，如果只有一个 tile 则不需要通过这个模块；如果当前计算只需要得到 psum，则不需要进行 BIAS、ReLU 和 Dequant。为了简化控制，直接提供一组 enable 信号来允许控制单元控制通过任意个子模块。

4.1 VPE

VPU 的每个 lane 的行为是相似的（但是不同步），因此将 VPU 设计先分解为单个 lane 的 Vector Processing Engine 设计。VPE 的设计如图12。

4.1.1 PSUM_Cache

本设计中，模型矩阵层大小无法完全放入一个 16x16 的 SystolicArray 阵列，因此需要做 tile，并且需要将多组部分和进行累加得到最终结果。此外，算法要求每次处理一个大小为 16 的批次。这些给 VPU 的控制带来一定的复杂度。

为了简化控制单元的设计，VPU 中包含了 PSUM_Cache，用于自动根据 SystolicArray 的 valid 输出，将数据输出自动累加到对应的位置，并在整个流程完成后统一输出。

VPU 设计中，为 PSUM_Cache 设计了 3 种模式，如图13：

1. Load 模式：如果是行优先进行切块，在某列 tile 计算完成后需要 psum 临时存回 unified buffer，在下一次计算本列 tile 时再将 psum 加载回 vpu。因此设计一个 load 模式用于将 psum 从 unified buffer 种加载到 vpu。对于列的第一次操作，或者是在列优先计算模式下，可以直接使用 psum_clear 信号清空 PSUM_Cache。
2. Accu 模式：正常的工作模式，VPU 的每个 lane 独立接收从 SystolicArray 送出的 valid & data，将其自动累加到对应的 cache 行。
3. Output 模式：累加结束后，需要将 psum 从 cache 种排出，将模式设定到 Output 模式，若干周期后数据从 VPU 输出。

4.1.2 BIAS & ReLU & Dequant

BIAS、ReLu 和 Dequant 模块较为简单，主要是进行算数运算。其中 Dequant 略微复杂，需要做浮点乘法。在实现种用 verilog 实现了一个简单的算数函数来完成。

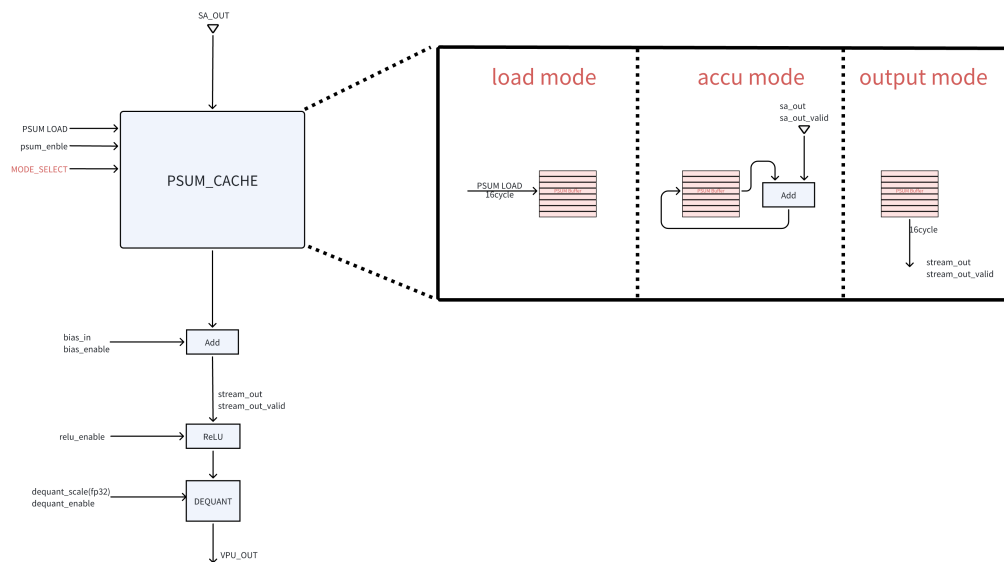


图 12: VPU 设计

4.2 VPU-Channel & VPU

将上述的 VPE 并行拼接，即可得到 VPU 的设计。

由于整体设计中包含 4 组 Systolic Array，这里分两个层次组合：先拼接 16xVPE 组成一个 VPU Channel，对应一个 Systolic Array 的输出。由于 4 组 Systolic Array 之间数据流模式是相同的，调试时可以专注调试一个 Channel。

随后将 4 个 Channel 组成 VPU 模块，如图14。

4.3 仿真

先介绍 VPU 三种模式的工作波形。为了方便，不失一般性地，只展示单个 vpe 的工作波形。

MODE_SELECT	
1	Accu模式, 逐次将有效的sa输出累加到对应缓冲区行中。默认batch size有16行
2	Load模式, 逐次将PSUM_LOAD_IN的BATCH_SIZE拍数据加载到对应的缓冲区行
3	Output模式, 逐次将PSUM buffer中的数据送出, 经过relu和dequant模块
0	invalid

图 13: vpe 模式说明

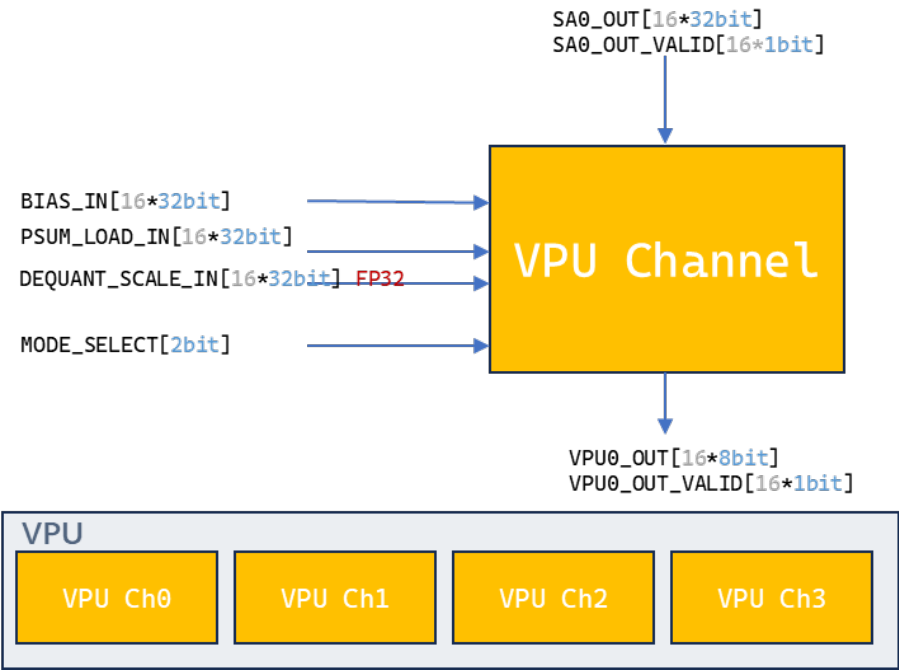


图 14: VPU-Channel 和 VPU 组织形式

首先是 Load 模式波形，如图15。在 Accu 模式下，将 mode_select 信号设为 Load 一个周期，进入 Load 模式。此后将连续从 psum_in 采样 16 拍输出分别存入 PSUM_Cache 的每一行。

Accu 模式下，如图16, vpu 自动检测 SystolicArray 的输出 valid 信号，将其输出累加到 psum_idx 选中的行，并更新 psum_idx。

Output 模式下，如图17，设定为 Output 模式，若干周期后，VPU 开始逐 cycle 排出缓存的 psum 结果。这时可选经过 BIAS、ReLU、Dequant 模块。



图 15: Load 模式波形

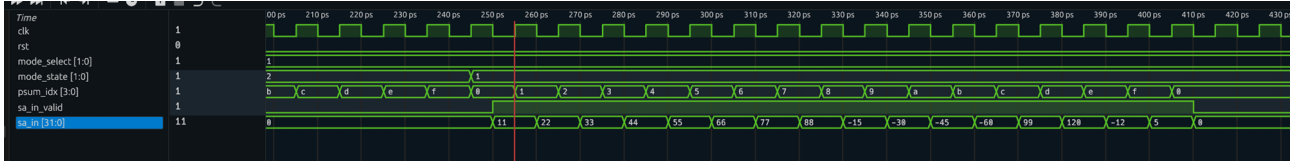


图 16: Accu 模式波形

5 SoC 集成与 DMA 设计

5.1 SoC 架构

在 TPU 设计的基础上，我们从系统层级进行扩展，其整体架构如图18所示。系统使用开源 RISC-V Ariane/CVA6 CPU¹作为 Host CPU，配置 128kB 的 SRAM 作为系统主存。TPU-Lite 通过 AXI 与存储接口转换模块 (axi2mem²模块) 相连，接入位宽为 64 比特的 AXI 系统总线。此外，基于开源 iDMA³ (Intelligent DMA)，我们实现 DMA Engine，将其集成在系统总线上，用于管理 TPU-Lite 与系统主存之间的数据搬运。

5.2 DMA Engine 设计

如图19所示，iDMA 采用模块化的设计，主要分为前端 (Front-end)，中端 (Mid-end) 与后端 (Back-end) 三个部分。其中，前端与中端模块为 DMA 的控制通路，负责发起数据传输请求，后端为数据通路，负责执行实际的数据搬运，并支持各类的数据传输接口，如 AXI，APB，RISC-V TileLink。

在本设计的 TPU-Lite-SoC 中，DMA Engine 的前端与后端均连接到 AXI 系统总线，其中前端为 AXI 从设备接口 (Slave Interface)，负责接收 CPU 的控制信号，实现 DMA 的配置；后端为 AXI 主设备接口 (Master Interface)，负责在 AXI 总线上发起数据传输。DMA 前端模块由若干寄存器组成，用于接收并存储 CPU 下发的配置信息，具体实现如代码6所示。

DMA 的配置工作通过 CVA6 Host CPU 实现，为此，我们编写相应的 C 代码，编译成 CPU 可执行程序，并存储到片上的系统主存中。DMA 配置通过代码7所示的函数实现，需要指定源地址 (src)，目的地址 (dst)，以及数据长度 (length_bytes) 三个参数信息。配置完成后，通过写入 DMA Launch 寄存器启动数据传输，随后等待传输完成。在这段示例代码中，CPU 主要角色为监控 DMA 数据传输状态；在实际系统设计中，CPU 可以在启动 DMA Engine 之后立刻执行其他的任务，无需等待所有数据传输完成，从而提升系统整体效率。

5.3 仿真与结果分析

为验证 DMA Engine 的数据传输功能，我们将其用于把数据从系统主存搬运到 TPU-Lite 上的 Unified buffer 中，单次传输的数据量为 64 比特，与 AXI 总线位宽一致，我们分别测试传输 16 个和 1024 个数据的场景，其仿真波形图如图20和图21所示。

¹<https://github.com/openhwgroup/cva6>

²https://github.com/pulp-platform/axi_mem_if

³<https://github.com/pulp-platform/iDMA>

```

1 always_ff @(posedge clk_i or negedge rst_ni) begin
2     if (!rst_ni) begin
3         src_addr_lo <= '0;
4         src_addr_hi <= '0;
5         dst_addr_lo <= '0;
6         dst_addr_hi <= '0;
7         length_lo <= '0;
8         length_hi <= '0;
9         config_reg <= '0;
10        launch_q <= 1'b0;
11    end else begin
12        launch_q <= 1'b0; // Auto-clear launch signal
13        if (req_i.valid && req_i.write) begin
14            unique case (req_i.addr[7:0])
15                8'h00: src_addr_lo <= req_i.wdata[31:0];
16                8'h08: src_addr_hi <= req_i.wdata[31:0];
17                8'h10: dst_addr_lo <= req_i.wdata[31:0];
18                8'h18: dst_addr_hi <= req_i.wdata[31:0];
19                8'h20: length_lo <= req_i.wdata[31:0];
20                8'h28: length_hi <= req_i.wdata[31:0];
21                8'h30: config_reg <= req_i.wdata[31:0];
22                8'h38: launch_q <= 1'b1;
23                default: ;
24            endcase
25        end
26    end
27 end
28 always_comb begin
29     rsp_o.ready = 1'b1;
30     rsp_o.error = 1'b0;
31     rsp_o.rdata = '0;
32     if (req_i.valid && !req_i.write) begin
33         unique case (req_i.addr[7:0])
34             8'h00: rsp_o.rdata = {32'b0, src_addr_lo};
35             8'h08: rsp_o.rdata = {32'b0, src_addr_hi};
36             8'h10: rsp_o.rdata = {32'b0, dst_addr_lo};
37             8'h18: rsp_o.rdata = {32'b0, dst_addr_hi};
38             8'h20: rsp_o.rdata = {32'b0, length_lo};
39             8'h28: rsp_o.rdata = {32'b0, length_hi};
40             8'h30: rsp_o.rdata = {32'b0, config_reg};
41             8'h40:
42                 rsp_o.rdata = {32'b0, 29'b0, ready_i, error_i, busy_i}; // Status: [2]=Ready,
43                                     [1]=Error, [0]=Busy
44             default: rsp_o.error = 1'b1;
45             endcase
46         end
47     end
48 end

```

Listing 6: DMA Engine 前端设计


```

1 #define REG32(base, offset)      (*((volatile uint32_t *)((uintptr_t)(base) + (offset))))
2
3 void dma_transfer(uint64_t src, uint64_t dst, uint32_t length_bytes) {
4     // Configure Source Address
5     REG32(DMA_ENGINE_BASE_ADDR, DMA_SRC_ADDR_LO_OFFSET) = (uint32_t)(src & 0xFFFFFFFF);
6     REG32(DMA_ENGINE_BASE_ADDR, DMA_SRC_ADDR_HI_OFFSET) = (uint32_t)(src >> 32);
7
8     // Configure Destination Address
9     REG32(DMA_ENGINE_BASE_ADDR, DMA_DST_ADDR_LO_OFFSET) = (uint32_t)(dst & 0xFFFFFFFF);
10    REG32(DMA_ENGINE_BASE_ADDR, DMA_DST_ADDR_HI_OFFSET) = (uint32_t)(dst >> 32);
11
12    // Configure Transfer Length
13    REG32(DMA_ENGINE_BASE_ADDR, DMA_LENGTH_LO_OFFSET) = length_bytes;
14    REG32(DMA_ENGINE_BASE_ADDR, DMA_LENGTH_HI_OFFSET) = 0;
15
16    REG32(DMA_ENGINE_BASE_ADDR, DMA_CONFIG_OFFSET) = 0x3;
17
18    // Launch Transfer
19    REG32(DMA_ENGINE_BASE_ADDR, DMA_LAUNCH_OFFSET) = 1;
20
21    // Fence to ensure DMA launch is seen
22    asm volatile ("fence" ::: "memory");
23
24    // Poll for Completion
25    while (REG32(DMA_ENGINE_BASE_ADDR, DMA_STATUS_OFFSET) & 0x1) {
26        // Wait while busy
27    }
28 }

```

Listing 7: DMA Engine 配置代码

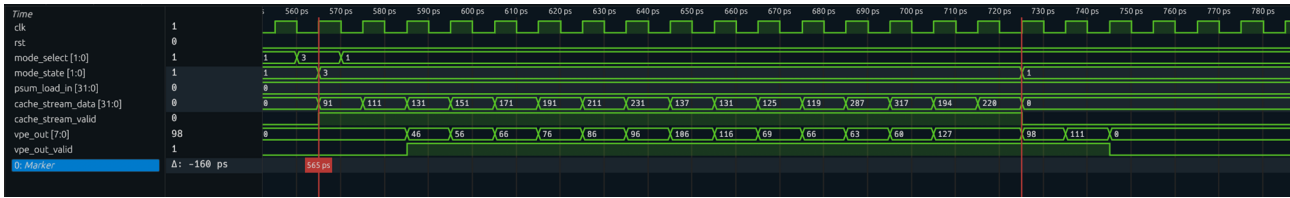


图 17: Output 模式波形

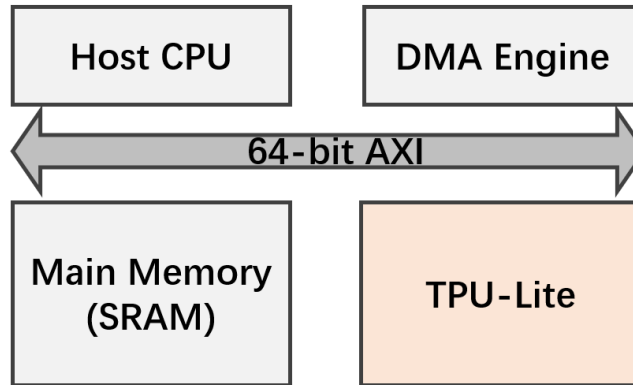


图 18: 系统层级架构

从波形中可以看到，DMA Engine 的后端模块先发起 AXI Burst 读请求，从系统主存（起始地址为 0x80001000）读取数据。数据经过 DMA Engine 内部的数据 FIFO，后端再向 TPU-Lite（起始地址 0x40000000）发起 AXI Burst 写请求。得益于 AXI 协议分离的读写通道，读操作与写操作可以并行进行，从而提升了数据传输的流水线效率。

对于大量的数据传输（例如 1024 个数据），其长度超过了 AXI 协议所允许的单次 Burst 传输最大长度（256 次）。因此 DMA Engine 会自动将此次传输任务分为 4 次独立的 AXI Burst 请求发送，如图21所示，体现了其处理长距离数据传输的稳定性。

5.4 性能对比：DMA 与 CPU 数据传输

为量化 DMA Engine 数据传输带来的性能优势，我们对比 DMA 和 CPU 执行相同数据传输时的延迟。测试的数据量范围从 1Kb 到 256Kb，通过记录任务执行周期数来对比传输延迟。我们使用 RISC-V 架构的 CSR 寄存器（如代码8所示）来精确获取 CPU 周期计数，并分别编写了使用 CPU 和 DMA 执行搬运的测试代码，如代码9所示。

性能对比结果如图22所示。随着传输数据量的增大，DMA Engine 带来的性能优势愈发明显。与 CPU 直接执行数据搬运相比，使用 DMA 可以获得 1.42× 到 14.68× 的性能提升。这体现了在 SoC 中集成专用 DMA 对于提升大数据搬运效率的必要性。

```

1 static inline uint64_t read_cycle() {
2     uint64_t cycle;
3     asm volatile ("rdcycle_%0" : "=r"(cycle));
4     return cycle;
5 }

```

Listing 8: CPU 记录执行周期

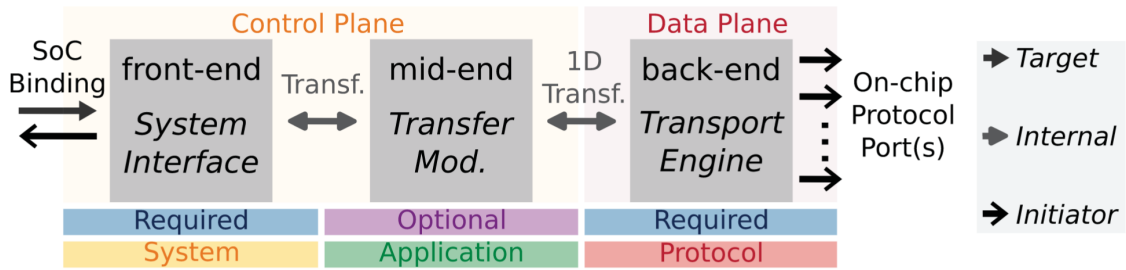


图 19: iDMA 各模块设计

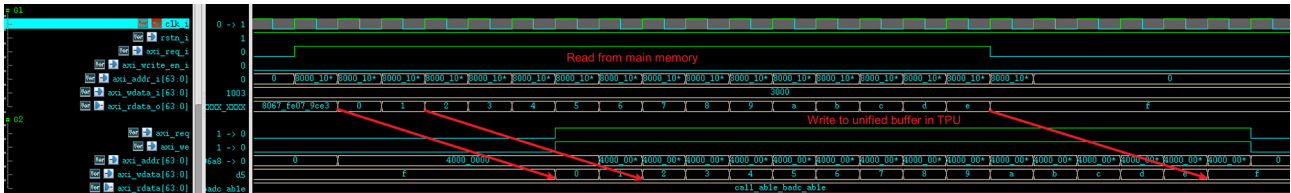


图 20: DMA Engine 数据传输 (16×64-bit)

6 软件层设计

软件层的核心目标具体包括：设计并训练 MLP 网络、完成 INT8 量化以适配硬件结构、开发与硬件推理过程完全对齐的 simulator、拆分计算流程以支撑指令设计与编译对接。最终为硬件模块提供模型输入和参数、golden result 及清晰的计算流程，保障 MNIST 数字分类任务的实现。

6.1 MLP 模型设计与训练

6.1.1 数据集预处理

原始 MNIST 数据集的单样本为 28×28 像素的灰度图像（像素值范围 0-255）。为适配 TPU-Lite-SoC 的硬件架构，设计如下预处理流程：1. 将 28×28 图像 resize 为 16×16 ，匹配 systolic array 的 16×16 计算粒度；2. 将 16×16 二维图像转换为 256 维一维向量，满足全连接层的输入格式要求；3. 将 UINT8 类型的像素值转换为 FP32 类型，并归一化至 $[0,1]$ 区间，消除不同像素维度的量纲差异，提升模型训练稳定性。

6.1.2 MLP 网络结构设计

结合硬件计算能力与分类任务需求，设计轻量化 MLP 网络，确保在保证分类精度的同时，最小化硬件资源占用。网络结构如图23所示：

输入层：维度为 $(bs, 256)$ (bs 为 batch size)；全连接层 1 (FC1)：Linear($256 \rightarrow 64$) + ReLU 激活函数，输出维度 $(bs, 64)$ ；全连接层 2 (FC2)：Linear($64 \rightarrow 32$) + ReLU 激活函数，输出维度 $(bs, 32)$ ；全连接层 (FC3)：Linear($32 \rightarrow 10$)，输出维度 $(bs, 10)$ （最大值对应数字分类结果）。

网络结构通过逐步缩减特征维度，平衡特征表达能力与计算复杂度，避免网络过深或维度冗余导致的硬件计算压力。

6.1.3 模型训练实现

基于 PyTorch 框架完成模型训练，采用 Adam 优化器，损失函数选择 CrossEntropyLoss 以适配多分类任务，训练批次设定为 30epochs。训练后 FP32 精度模型在测试集上的准确率²⁴达到 97.43%，同时保存模型权重文件用于后续量化流程。

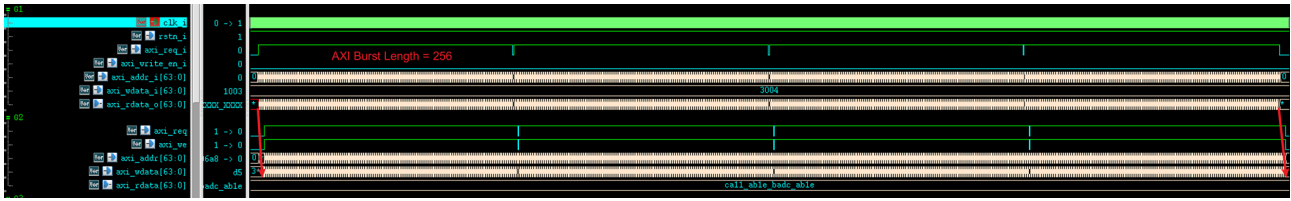


图 21: DMA Engine 数据传输 (1024×64-bit)

```

1 // CPU Write (Main -> UBUF)
2 uint64_t start = read_cycle();
3 for (int j = 0; j < size; j++) {
4     ubuf[j] = main_mem[j];
5 }
6 uint64_t end = read_cycle();
7 *log_ptr++ = end - start;
8 *error_flag = 0x2000 + i;
9
10 // DMA Write (Main -> UBUF)
11 start = read_cycle();
12 dma_transfer(MAIN_MEM_BASE_ADDR, UBUF_START_ADDR, bytes);
13 end = read_cycle();
14 *log_ptr++ = end - start;
15 *error_flag = 0x4000 + i;

```

Listing 9: DMA 与 CPU 数据传输配置代码

6.2 INT8 量化

量化的核心目标是将 FP32 精度模型转换为 INT8 精度，适配硬件的计算单元。量化遵循以下原则：

1. 采用 uniform 量化方案，zero-point=0，简化量化逻辑；
2. 权重、输入、激活分别独立量化；
3. 偏置量化采用 INT32 精度，适配矩阵乘累加过程中采用的数据格式；
4. 生成合并系数整合缩放因子，减少硬件存储的参数数量。

6.2.1 量化流程与实现细节

量化流程基于训练完成的 FP32 模型，依次完成权重、输入、偏置、激活的量化及合并系数生成，具体实现如下：

1. **权重量化 (FP32→INT8)** 针对每一层全连接层的权重，采用基于最大值的对称量化方法：计算权重矩阵的最大绝对值 max_val ；计算缩放因子 $scale = max_val/127.0$ (127 为 INT8 的最大正值)；将 FP32 权重归一化后四舍五入为 INT8，并裁剪至 $[-127, 127]$ 范围，避免溢出。

量化后输出 3 层权重，维度分别为： $W_1(256,64)$ 、 $W_2(64,32)$ 、 $W_3(32,10)$ ，数据类型均为 INT8。

2. **输入量化 (FP32→INT8)** 统计测试集所有预处理后输入数据的最大绝对值 $full_input_max$ ；计算输入缩放因子 $input_scale = full_input_max/127.0$ ；对所有展平后的 256 维输入向量执行量化操作，输出数据类型为 INT8，维度为 $(N, 256)$ (N 为样本数)。

3. **偏置量化 (FP32→INT32)** 偏置量化需匹配矩阵乘结果的精度 (INT8)，避免累加过程中的精度损失：偏置缩放因子 $scale_b = 输入缩放因子 \times 对应层权重缩放因子$ ；将 FP32 偏置除以 $scale_b$ 后四舍

```
==== TPU Transfer Speed Comparison Results ====
Size(words) | CPU Write | CPU Read | DMA Write | DMA Read
16 | 218 | 273 | 153 | 106
64 | 680 | 973 | 149 | 135
256 | 2672 | 3853 | 330 | 330
512 | 4752 | 7693 | 585 | 585
1024 | 9497 | 15373 | 1095 | 1095
2048 | 22767 | 30733 | 2130 | 2130
4096 | 47012 | 61453 | 4185 | 4185
==== Test Complete ====
```

图 22: DMA 与 CPU 数据传输对比

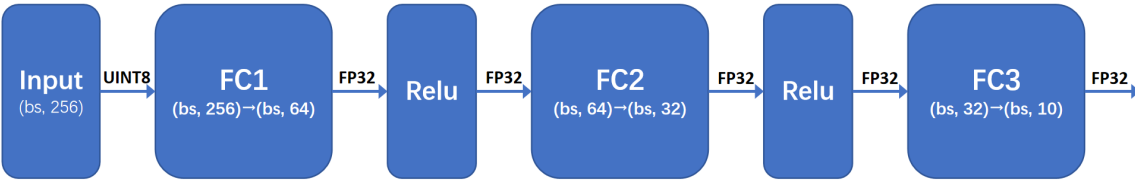


图 23: MLP 网络结构 (FP32 训练阶段)

五入为 INT32 类型；输出 3 层偏置，维度分别为： $b_1(1,64)$ 、 $b_2(1,32)$ 、 $b_3(1,10)$ ，数据类型均为 INT32。

4. 激活缩放因子与合并系数生成 激活缩放因子：采样 10,000 个测试样本，通过前向传播收集各线性层输出 (FP32)，计算每层输出的最大绝对值，生成 3 个激活缩放因子 (scale1、scale2、scale3，均为 FP32)；合并系数25：为简化硬件计算，整合输入、权重、激活的缩放因子，生成合并系数 $k = \text{输入 scale} \times \text{权重 scale} / \text{激活 scale}$ ，共 3 个（对应 3 层全连接层），数据类型为 FP32。

6.2.2 量化参数输出

表 2: INT8 量化输出参数列表

参数名称	维度	数据类型	说明
quantized_weight_0	(256, 64)	INT8	第 1 层全连接层权重
quantized_weight_1	(64, 32)	INT8	第 2 层全连接层权重
quantized_weight_2	(32, 10)	INT8	第 3 层全连接层权重
quantized_bias_0	(64,)	INT32	第 1 层全连接层偏置
quantized_bias_1	(32,)	INT32	第 2 层全连接层偏置
quantized_bias_2	(10,)	INT32	第 3 层全连接层偏置
quantized_input	(N, 256)	INT8	量化后输入数据
merge_coefficients	(3,)	FP32	硬件量化缩放合并系数

6.3 硬件 INT8 推理 simulator

为支撑硬件模块的功能验证与 debug，需要开发与硬件推理过程完全对齐的 INT8 推理仿真器，核心目标包括：功能一致性：复现硬件从输入到输出的全链路推理流程，包括矩阵乘、偏置加、量化缩放、

```
Epoch 26/30 - Loss: 21.7009
Epoch 27/30 - Loss: 21.2331
Epoch 28/30 - Loss: 20.0495
Epoch 29/30 - Loss: 19.3356
Epoch 30/30 - Loss: 18.5266
测试集准确率: 97.43%
```

图 24: 测试集准确率

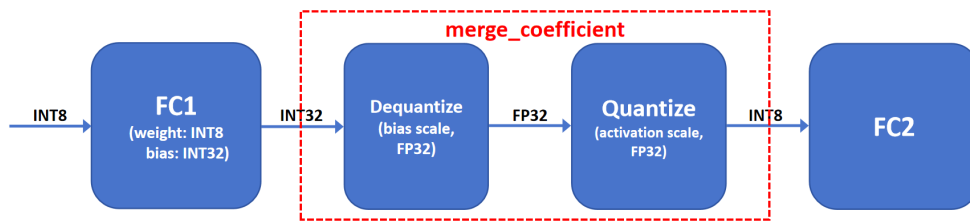


图 25: 合并系数 merge_coefficient

ReLU 激活等所有操作；精度匹配性：确保仿真器输出的中间结果（如矩阵乘结果、量化后数据）与硬件计算结果的数值精度完全一致；调试支撑性：支持 batch size=16（与硬件设计一致），可灵活输出任意中间节点的 golden result，为硬件模块（systolic array、VPU）提供调试基准。

6.3.1 仿真核心流程

仿真器严格遵循硬件推理逻辑，图26中以 Layer1 为例，展示了推理的核心流程与数据类型转换。

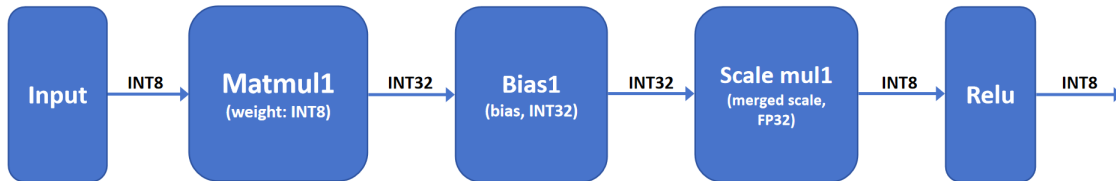


图 26: INT8 推理流程 (Layer1)

具体流程：1. 输入层：接收 INT8 量化输入数据，batch size 设置为 16，维度为 (16,256)；2. 第 1 层计算：矩阵乘：INT8 输入 $\times$ INT8 权重 $\rightarrow$ INT32 结果（维度 (16,64)）；偏置相加：INT32 矩阵乘结果 + INT32 偏置 $\rightarrow$ INT32 结果；量化缩放：INT32 结果 $\times$ FP32 合并系数 $\rightarrow$ INT8 结果；ReLU 激活：维度 (16,64)；3. 第 2 层计算：重复第 1 层流程，输出 INT8 数据（维度 (16,32)）；4. 第 3 层计算：执行矩阵乘、偏置加、量化缩放，不执行 ReLU 激活，输出 INT8 分类结果（维度 (16,10)）。

6.3.2 仿真器实现与验证结果

仿真器核心实现代码如下10：

仿真器验证结果：精度验证：测试集 INT8 推理准确率27达到 97.56%。debug 支持：可输出任意中间结果，例如第 1 层 ReLU 激活后的 INT8 数据、第 2 层矩阵乘的 INT32 累加结果等，为硬件模块 debug 提供直接参考。

```

1 def mlp_int8_inference(input_int8, quant_params):
2     """INT8量化推理核心逻辑（与硬件对齐）"""
3     merge_coeffs = quant_params["merge_coefficients"]
4     weights = quant_params["quantized_weights"]
5     biases = quant_params["quantized_biases"]
6
7     current_input = input_int8 # (16, 256) INT8
8
9     for layer_idx in range(3):
10        # 1. 矩阵乘 (INT8×INT8→INT32)
11        w_int8 = weights[layer_idx] # (out_dim, in_dim) INT8
12        matmul_res = np.dot(current_input.astype(np.int32), w_int8.T.astype(np.int32))
13
14        # 2. 偏置加 (INT32+INT32→INT32)
15        b_int32 = biases[layer_idx]
16        add_res = matmul_res + b_int32[np.newaxis, :]
17
18        # 3. 合并系数缩放 (INT32×FP32→INT8)
19        merge_res = add_res * merge_coeffs[layer_idx]
20        output_int8 = merge_res.astype(np.int8)
21
22        # 4. 激活函数（最后一层无ReLU）
23        if layer_idx == 2:
24            return output_int8
25        current_input = np.maximum(output_int8, 0) # ReLU: 负数置0
26
27    return None
28
29 def main():
30     # 加载参数与数据
31     quant_params = np.load("mlp_int8_quantized.npz")
32     input_int8 = quant_params["quantized_input"]
33     test_labels = np.load("mnist16_test_labels.npy")
34
35     # 批量推理 (batch size=16)
36     batch_size = 16
37     num_samples = len(input_int8)
38     predictions = []
39     for start in range(0, num_samples, batch_size):
40         end = min(start+batch_size, num_samples)
41         batch_out = mlp_int8_inference(input_int8[start:end], quant_params)
42         predictions.extend(np.argmax(batch_out, axis=1))
43
44     # 精度验证
45     accuracy = np.mean(np.array(predictions) == test_labels) * 100
46     print(f"INT8推理准确率: {accuracy:.2f}%")

```

Listing 10: INT8 推理 simulator 核心代码


```

开始INT8推理...
加载量化参数...

推理完成！
测试集样本数：10000
INT8量化推理准确率：97.56%

```

图 27: INT8 推理 simulator 输出结果

6.4 计算流程拆分与指令设计对接

为支撑指令设计与编译任务，基于 TPU-Lite-SoC 的硬件架构，对 MLP 推理计算流程进行精细化拆分。按 16×16 维度拆分矩阵乘法，匹配 systolic array 的计算单元尺寸；明确各计算阶段数据的维度、数据类型及存储位置，确保与 Unified Buffer、DMA 的数据交互格式一致；拆分后的步骤与硬件指令一一对应，便于编译生成指令流；明确各步骤的输入输出依赖关系，避免指令执行冲突。

6.4.1 分层计算流程拆分

以第 1 层全连接层（输入 $(16, 256) \rightarrow$ 输出 $(16, 64)$ ）为例，详细拆分流程如下：

1. 输入与权重矩阵分块 输入矩阵分块：将 $(16, 256)$ 的 INT8 输入矩阵拆分为 16 个 $(16, 16)$ 的子矩阵，记为 $[A_1, A_2, \dots, A_{16}]$ ；权重矩阵分块：将 $(256, 64)$ 的 INT8 权重矩阵拆分为 16×4 个 $(16, 16)$ 的子矩阵，如下式：

$$W_1 = \begin{bmatrix} W_{11} & W_{12} & W_{13} & W_{14} \\ W_{21} & W_{22} & W_{23} & W_{24} \\ \vdots & \vdots & \vdots & \vdots \\ W_{15,1} & W_{15,2} & W_{15,3} & W_{15,4} \\ W_{16,1} & W_{16,2} & W_{16,3} & W_{16,4} \end{bmatrix}$$

2. 分步计算逻辑

- 第 1 步：Systolic Array 计算 $A_1 \times W_{11}$ 、 $A_1 \times W_{12}$ 、 $A_1 \times W_{13}$ 、 $A_1 \times W_{14}$ ，结果（INT32）存入 VPU；
- 第 2 步：Systolic Array 计算 $A_2 \times W_{21} \sim A_2 \times W_{24}$ ，VPU 接收到结果后执行累加运算： $A_1 W_{11} + A_2 W_{21}$ 、 $A_1 W_{12} + A_2 W_{22}$ 、 $A_1 W_{13} + A_2 W_{23}$ 、 $A_1 W_{14} + A_2 W_{24}$ ，结果（INT32）覆盖保存；
- 第 3-16 步：重复第 2 步逻辑，依次计算 $A_3 \sim A_{16}$ 与对应权重子矩阵的乘积，VPU 持续累加，最终得到 $(16, 64)$ 的 INT32 矩阵乘结果；
- 第 17 步：VPU 读取偏置数据（INT32），与矩阵乘结果逐行相加，输出 $(16, 64)$ 的 INT32 数据；
- 第 18 步：VPU 加载合并系数（FP32），与上述结果逐元素相乘，量化为 INT8 数据；
- 第 19 步：VPU 执行 ReLU 激活，将 INT8 数据中负数置 0；
- 第 20 步：VPU 将激活后的数据 $((16, 64), \text{INT8})$ 写回 Unified Buffer，作为第 2 层输入。

第 2 层 $((16, 64) \rightarrow (16, 32))$ 和第 3 层 $((16, 32) \rightarrow (16, 10))$ 的拆分逻辑与第 1 层一致，仅分块数量和输出维度适配对应层参数。

6.4.2 指令对接支持

为指令设计同学提供以下关键信息，支撑编译流程开发：操作类型：矩阵乘（ 16×16 ）、累加、偏置相加、量化缩放、ReLU 激活、数据写回；数据维度映射：明确各操作的输入/输出张量维度、数据类型（INT8/INT32/FP32）；指令依赖关系：例如“累加操作”依赖前一次累加结果和当前矩阵乘结果，“量化缩放”依赖偏置加结果和合并系数。

7 功能验证与评估

在完成 TPU-Lite-SoC 各模块设计与验证后，我们集成各个模块进行设计验证，旨在确保 RTL 设计符合架构设计规范，能够正确执行指令加载、权重预取、脉动阵列计算及 VPU 后处理，并通过 AXI 总线正确交互。

验证使用 Synopsys VCS 仿真器，配合 Verdi 进行波形调试。Testbench 采用 SystemVerilog 编写。

7.1 TPU-Lite 功能验证

首先我们集成 TPU-Lite 各个模块，进行功能验证。验证过程模拟了 TPU 的完整工作生命周期，包含四个关键阶段，**初始化与数据加载**，**指令读取与解码**，**核心计算**，以及 **VPU 后处理与写回**。

7.1.1 测试平台架构

验证环境的拓扑结构如下：

- **DUT (Device Under Test)**: TPU 顶层模块，包含 AXI 接口、控制单元、统一缓冲区 (Unified Buffer)、指令缓存 (I-Cache)、 $4 \times 16 \times 16$ 脉动阵列及 VPU。
- **Clock/Reset Gen**: 生成 100MHz 时钟信号及同步复位信号。
- **AXI Master VIP (Simplified)**: 模拟 CPU 行为，支持 AXI 协议的读写时序，负责向 TPU 存储器写入数据/指令，并配置状态寄存器。
- **Monitor/Checker**: 实时监控 AXI 总线上的 `axi_rdata`，并自动比对计算结果与预期黄金模型 (Golden Model)。

7.1.2 内存映射 (Memory Map)

根据 RTL 设计 `axi_interface.sv`，验证环境严格遵循以下地址映射：

表 3: TPU AXI 地址空间映射

模块	基地址 (Offset)	绝对地址 (Base 0x40000000)	功能描述
Unified Buffer (UB)	0x0000 - 0x29FF	0x40000000	数据/权重存储
Instruction Cache	0x2A00 - 0x2DFF	0x40015000	54-bit 指令存储
Status Register	0x2E00 - 0x2E01	0x40017000	全局使能与完成标志

7.1.3 阶段一：初始化与数据加载

测试内容：验证 AXI 接口对内部存储器的写入能力。

1. **指令加载**: Testbench 通过 AXI 总线将编译好的二进制机器码写入 I-Cache 区域。验证确认 `axi_icache_we` 信号正确拉高, 数据正确写入 `instruction_cache`。
2. **权重与数据加载**: 将输入特征图 (Feature Map) 和权重矩阵分别写入 UB 的 Input Memory 区 (Offset 0x0000) 和 Weight Memory 区 (Offset 0x2000)。波形显示 `input_memory` 与 `weight_memory` 均已正确初始化。

7.1.4 阶段二: 指令读取与解码

测试内容: 验证 `control_unit` 的取指译码逻辑。

- 配置状态寄存器 `REG_ENABLE` (Addr 0x40017000) 为 1 后, 全局使能信号 `global_en` 拉高。
- 观察到 `icache_rd_ctrl_addr` 从 0 开始递增, 54-bit 指令被正确读出并解码为控制信号 (如 `ins_sa_en_size`, `ins_vpu_mode_select` 等)。

7.1.5 阶段三: 核心计算 (Weight Stationary 数据流)

测试内容: 验证脉动阵列的权重预加载机制及矩阵乘法。

- **权重预加载**: 在执行计算指令前, 观察到 `sa_new_weight` 信号有效。数据经过 `systolic_data_rearranger` 进行移位打拍, 以阶梯状时序进入阵列, 符合权重驻留 (Weight Stationary) 架构的时序要求。
- **脉动计算**: 输入数据 `sa_input` 从阵列左侧进入, 部分和在阵列内部累加。观察到 `sa_output` 在延迟若干周期后输出有效的 32-bit 累加结果。

7.1.6 阶段四: VPU 后处理与写回

测试内容: 验证向量处理单元的非线性变换及结果回写。

- **Bias 与 ReLU**: VPU 正确读取了 Mmem 中的 Bias 数据, 并根据指令中的 `vpu_relu_enable` 标志位执行了激活操作。
- **结果回写**: VPU 输出的 8-bit 结果通过 `ub_wr_VPU_data_in` 端口写回 UB 的 Input Memory 区域, 实现了层间数据的循环利用。
- **完成中断**: 计算结束后, 状态寄存器的 `finish_flag` (Addr 0x40017008) 被置为 1, Testbench 成功捕获该信号并终止仿真。

7.1.7 波形验证

使用训练得到的 MLP 模型的第一层权重以及真实输入数据来进行验证, 权重矩阵的储存为 64 行 256 列, 输入特征矩阵 (batch size 为 16) 的尺寸为 16 行 256 列。由于脉动阵列一次可以完整计算的最大矩阵尺寸为 16x16 输入矩阵和 16x64 权重矩阵乘法, 因此需要 16 次矩阵乘计算并且在 VPU 中将每次的结果累加。

最终得到的计算结果和软件仿真的 Golden Results 相符。偶尔会有相差 1 比特的情况, 可能是硬件端的舍入与软件端行为的差异, 但是在合理误差范围内。

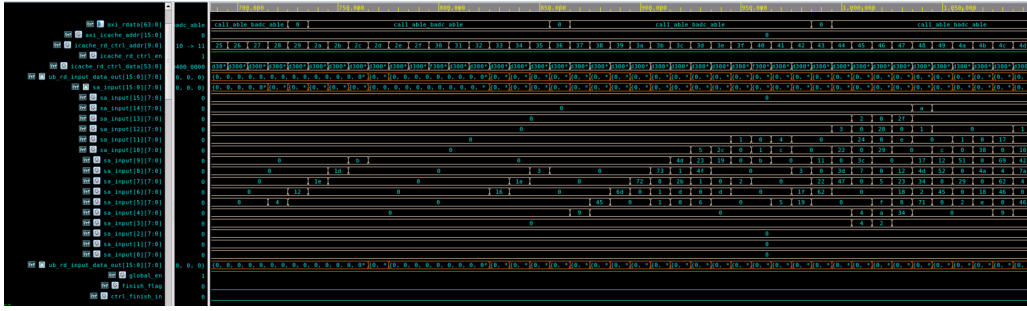


图 28: TPU 中 SA 的 input 矩阵输入

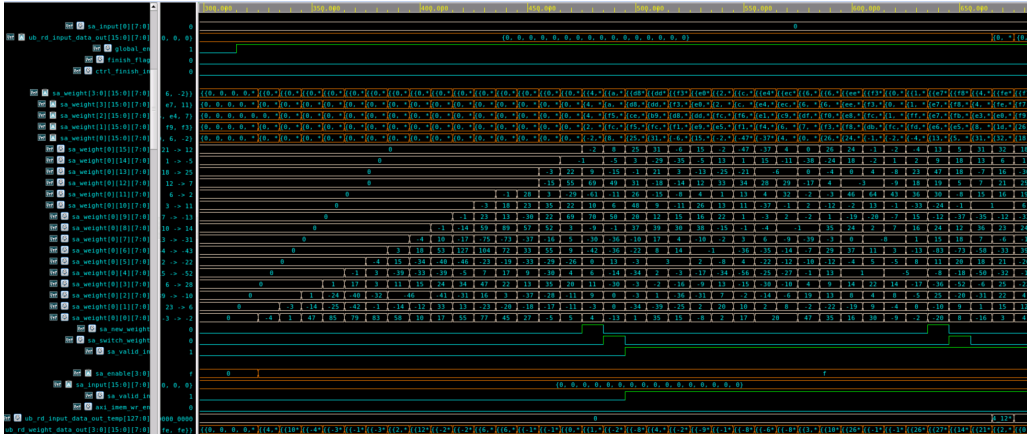


图 29: TPU 中 SA 的 weight 矩阵输入

7.2 TPU-Lite-SoC 系统级功能验证

在完成 TPU 模块级验证的基础上，我们进一步在完整的 SoC 环境中进行了系统级功能验证。验证流程通过编写 C 代码并由 CVA6 CPU 执行驱动，其步骤与章节5中描述的仿真流程基本一致。

具体而言，C 程序首先通过调用 `dma_transfer` 函数，将 TPU 指令、模型权重以及输入激活值等数据从系统主存搬运至 TPU 的全局缓冲区（Global Buffer）中。随后，CPU 通过配置寄存器启动 TPU 运算。在整个计算过程中，CPU 持续轮询 TPU 的状态寄存器以监控其运行状态。待 TPU 计算完成后，CPU 再将全局缓冲区中的计算结果读出，并与预先生成的 Golden Results（Golden Results）进行比对，以验证计算的正确性。

系统执行的整体流程波形如图32所示，清晰地展示了从 DMA 数据搬运、TPU 执行到 CPU 验证结果的完整时序。执行 Testbench 输出结果如图33所示。

7.2.1 CPU 执行硬件结果验证机制

为便于在仿真波形中直观地检查并比对每一份计算结果，使用 CPU 通用寄存器辅助验证。我们将 Golden Results 值与 TPU 的实际输出值分别写入 CVA6 CPU 的 S3 与 S4 寄存器。由于 CVA6 CPU 未采用寄存器重命名技术，其 32 个物理寄存器与 RISC-V 架构规范中的寄存器一一对应，并且我们所编写的验证代码并未使用到被调用者保存寄存器（S0-S11），因此该方法可靠且不会引入副作用的。

我们将结果不匹配（Mismatch）的数量实时累加并写入 S2 寄存器，并在验证流程的最后，通过向 S2 寄存器写入特定的标志值（如 0xABAB 表示成功，0xDEAD 表示失败）来汇总最终验证结论。考虑到软件层面的浮点计算或量化处理与 TPU 硬件执行之间可能存在细微的数值差异，我们在比对时允许最多 1 比特的误差。验证流程的核心代码如代码11所示。

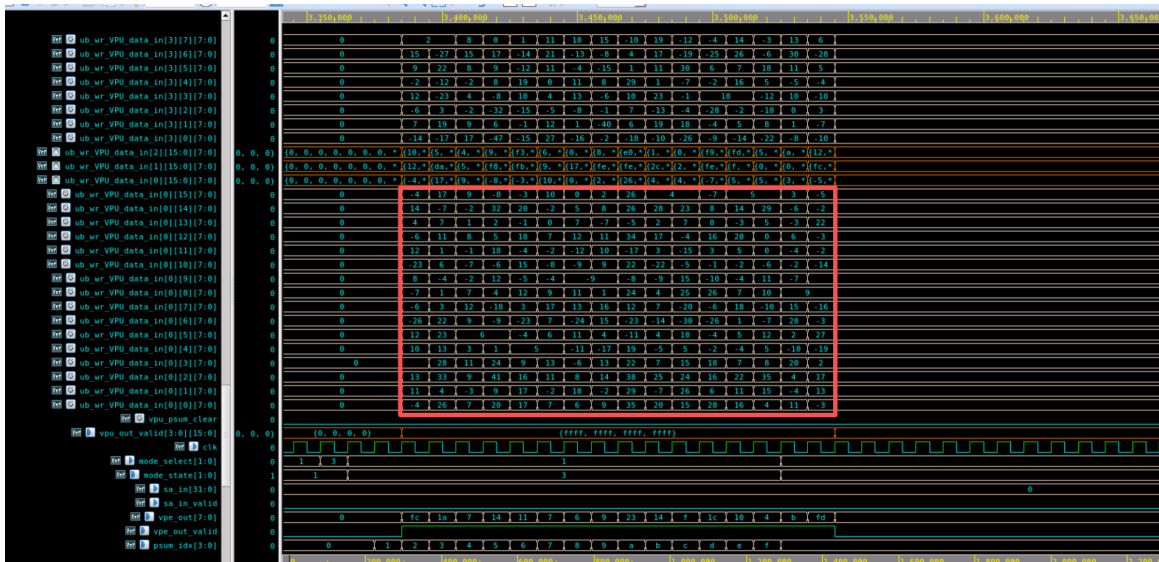


图 30: TPU 计算结果

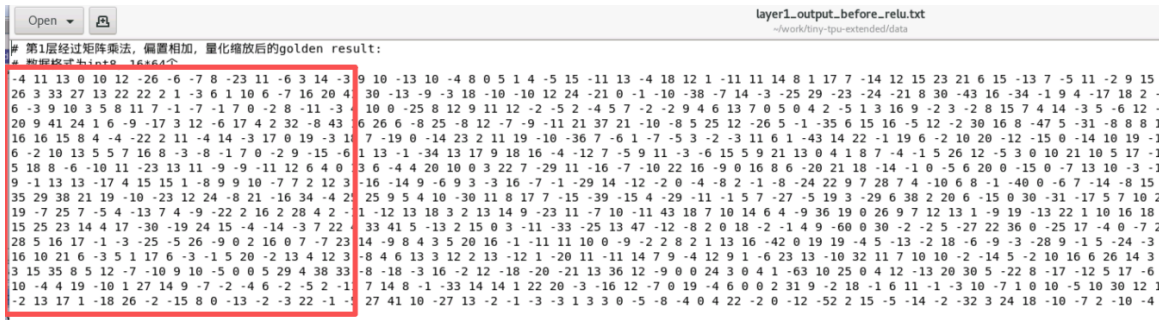


图 31: 软件仿真 Golden Results

7.2.2 验证结果分析

通过观察仿真波形中 CPU 寄存器文件的变化 (如图34所示), 可以清晰地看到 S3 寄存器 (存储 Golden Results) 与 S4 寄存器 (存储 TPU 输出) 的值在整个验证过程中始终保持高度一致, 其差异始终控制在预先设定的 1 比特误差范围内。这直接证明了 TPU-Lite 在集成到 SoC 系统后, 其计算功能正确无误。

最终, S2 寄存器被正确地写入成功标志 “0xABAB”, 标志着所有测试向量均通过验证, TPU-Lite-SoC 系统功能完整且符合设计预期, 如图35所示。

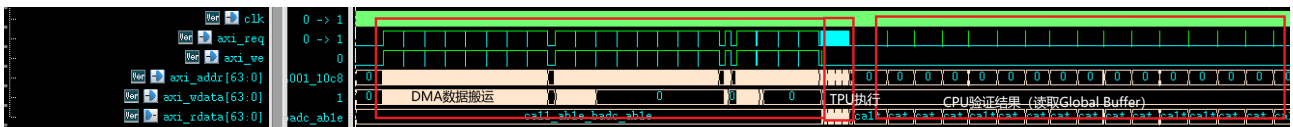


图 32: SoC 系统级 TPU 执行仿真波形

```
=== Checking MLP Inference Results ===
Inference results match golden results!
=== Test Complete ===
```

图 33: SoC 系统级 TPU 执行结果输出



图 34: CPU S3、S4 寄存器值变化波形 (用于结果比对)



图 35: CPU S2 寄存器值变化波形 (示意验证结果正确)

```

1 int errors = 0;
2 volatile uint64_t* result_base_64 = (volatile uint64_t*)(uintptr_t)(TPU_BASE_ADDR + 0
    x11000);
3 for (int i = 0; i < OUTPUT_ROWS; i++) {
4     for (int w = 0; w < 8; w++) {
5         uint64_t word_val = result_base_64[i * 8 + w];
6         for (int b = 0; b < 8; b++) {
7             int8_t output_value = (int8_t)((word_val >> (b * 8)) & 0xFF);
8             int byte_offset_in_row = w * 8 + b;
9             int chunk_idx = 3 - (byte_offset_in_row / 16); // 3->0, 2->1, 1->2, 0->3
10            int byte_in_chunk = byte_offset_in_row % 16;
11            int logical_j = chunk_idx * 16 + byte_in_chunk;
12            int8_t expected_value = golden_result[i * OUTPUT_ROW_SIZE + logical_j];
13            asm volatile ("mv▯s3,▯%0" :: "r"(expected_value));
14            asm volatile ("mv▯s4,▯%0" :: "r"(output_value));
15            if (output_value != expected_value && output_value != (expected_value + 1) &&
                output_value != (expected_value - 1)) {
16                errors++;
17                asm volatile ("mv▯s2,▯%0" :: "r"(errors));
18            }
19        }
20    }
21 }
22 if (errors == 0) {
23     // Write 0xABAB to s2 for success
24     asm volatile ("li▯s2,▯0xABAB");
25 }
26 else {
27     // Write 0xDEAD to s2 for failure
28     asm volatile ("li▯s2,▯0xDEAD");
29 }

```

Listing 11: CPU 验证计算结果核心代码